

Aparajita: Branchless SIMD Search and Append-Only Nodes for LSM-Tree MemTables

Parth Kumar Sinha
Independent Researcher
Oxford, United Kingdom
ORCID: 0009-0002-3120-9301

Abstract—The in-memory write buffer of an LSM-tree key-value store is searched on every read and written on every insert, so its cost per operation sets a floor on the whole engine. RocksDB uses a concurrent skip list, which chases pointers across independently allocated nodes and evaluates one data-dependent branch per key comparison. We present Aparajita, a MemTable representation that replaces the skip list with a list of cache-line-sized nodes, each holding fifteen 32-bit order-preserving key surrogates and a sentinel in one line, searched by a branchless SIMD kernel. Three design decisions carry the result. A relational vector compare over a sorted node yields a mask whose population count is the lower bound directly, so ordered search costs one compare, one movemask and one popcount with no branch. Surrogates are taken after the node’s shared prefix rather than from the start of the key, without which an absolute surrogate takes one value across all 200,000 keys in five of eight realistic distributions, including the keyspace this paper’s own evaluation runs on. Nodes are append-only, and the sorted order over their slots is a 64-bit word, so an insert is two stores into a free slot followed by one release store that publishes them.

We implement Aparajita as a RocksDB plugin selectable by name without patching RocksDB sources, and evaluate it against the default skip list and VectorRep on a 12-core Emerald Rapids host. Point lookups over a resident MemTable are 22% to 39% faster than the skip list at 1, 4, 16 and 64 threads, non-overlapping across five runs per configuration at every point but one, backed by 55% fewer retired instructions and 40% fewer L1 misses per lookup. Ordered seeks are 15% to 29% faster, but a seek followed by ten iterator steps is 3% to 4% slower, and the representation charges 1.4 times the skip list’s arena per key. The skip list is 48% to 52% slower on insert at the representation level, and Aparajita is 18.8% faster in single-threaded `db_bench`, but the multi-threaded `db_bench` write path is bounded by RocksDB’s write group rather than by the MemTable: an insert there retires over 22,000 instructions in both representations. We report that ceiling rather than a write scaling claim the data does not support.

Index Terms—LSM-tree, key-value store, MemTable, SIMD, branch prediction, cache-conscious data structures, RocksDB

I. INTRODUCTION

An LSM-tree [1], [2] absorbs writes into an in-memory buffer and flushes it to sorted files on disk when it fills. In RocksDB [3]–[5] and in LevelDB [6], from which it descends, that buffer is the MemTable, and every read consults it before any file, so it sits on both the write path and the read path of the entire engine. Its default implementation is a concurrent skip list [7].

A skip list is a good fit for the interface and a poor fit for the hardware. Each node is separately allocated, so a search follows a chain of pointers whose targets are unrelated in

memory, and each hop costs a cache miss that the prefetcher cannot anticipate because the address is not known until the previous load returns. Each hop also compares one key against one bound, which is a data-dependent branch, and the branch is unpredictable by construction: the whole point of the comparison is that its outcome is not known in advance. Modern cores are built to hide latency through parallelism, and a dependent chain of misses followed by mispredicted branches defeats both mechanisms at once.

The alternative this paper explores is to make the unit of search a single cache line. If sixteen 32-bit lanes fit in one 64-byte line and can be compared in one or two vector instructions, then a search over the keys they hold costs one cache fill, no pointer chase, and no branch at all. Reaching the right line still requires a descent, so the design also has to make the descent cheap; a fast leaf reached by an expensive walk is not an improvement.

Getting from that idea to a working RocksDB representation raises three problems that shape the rest of the paper.

RocksDB keys are variable-length internal keys, not 32-bit integers, so a vector lane holds a surrogate rather than a key. The surrogate has to preserve order, or the same kernel cannot serve both point lookups and the ordered iteration that RocksDB requires. It also has to discriminate, which turns out to rule out the obvious choice of leading key bytes.

An ordered structure with fixed-capacity nodes needs an insert that keeps the node sorted, and shifting keys inside a node is not a single atomic operation. A design that publishes each insert through one release store therefore has to separate the key material from the order over it.

The measurement itself is a problem. A MemTable representation is measured through a benchmark harness that also exercises a write-ahead log, a block cache, a version set and a write-thread group, and several of those cost more per operation than the structure under test. Part of this paper’s contribution is identifying where the harness, rather than the representation, is what a number describes.

A. Contributions

- A cache-line node layout with a branchless ordered SIMD search kernel whose population count is the lower bound directly, measured at 4.03 cycles per probe on AVX-512 and 5.10 on AVX2 against 40.14 for the branchy scalar

lower bound it replaces, with zero branch mispredictions per probe.

- A prefix-relative order-preserving surrogate, motivated by a measurement showing that an absolute leading-bytes surrogate takes a single value across all 200,000 keys in five of eight realistic distributions, the keyspace the evaluation itself uses among them.
- An append-only node whose sorted order is a 64-bit word of packed slot indices, making an insert one release store and reducing arena consumption from 445.4 to 100.6 bytes per key.
- An eight-byte descent hint in the node header that replaces a virtual comparator call per tower hop with an integer compare in a line already loaded.
- An evaluation against RocksDB’s skip list and VectorRep covering point lookups, ordered seeks, inserts, arena cost and flush cost, that separates the representation’s cost from the harness’s, including two measurement errors we made and corrected and an exit criterion we could not meet.

II. BACKGROUND AND RELATED WORK

A. MemTable representations in RocksDB

RocksDB exposes the MemTable through the MemTableRep interface and ships several implementations. The default SkipListRep wraps a concurrent skip list supporting lock-free reads and concurrent inserts. VectorRep appends to a `std::vector` and sorts when the MemTable becomes immutable, which makes its inserts very cheap and its reads on a mutable buffer very expensive, because a point lookup on an unsorted vector has to sort a copy. HashSkipListRep and HashLinkListRep bucket by prefix and are useful only when the workload’s access pattern matches the prefix extractor.

A reviewer familiar with RocksDB will ask why Aparajita is not VectorRep with vector instructions. The answer is that VectorRep gives up ordering, and ordering is what a MemTable iterator has to provide. Section V-G measures what that costs VectorRep in practice, and Section V-D measures what providing it costs us.

B. Cache-conscious and SIMD search

Making search structures fit the memory hierarchy is long-established. Rao and Ross showed that removing pointers from B^+ -tree nodes improves main-memory search [8]. FAST laid out a tree so that its levels match cache line, page and SIMD widths [9]. Schlegel et al. generalised binary search to k -ary search driven by vector comparisons [10], and Zeuch et al. applied similar transformations to tree traversal [11]. Masstree combines a trie of B^+ -trees with careful attention to cache line behaviour [12], the Adaptive Radix Tree adapts node representations to density [13], and the Bw-Tree reaches latch freedom through a mapping table and delta records [14].

Masstree deserves separating out, because its keyslice is the closest prior art to two of the mechanisms here. Masstree cuts a variable-length key into fixed eight-byte slices and compares

a slice as an integer, falling back to the full key when slices tie, which is structurally what both our surrogate and our descent hint do. The difference is where the fixed window sits. A Masstree slice is taken at a depth fixed by the trie level a node occupies, so the bytes it covers are the same for every node at that depth. Our surrogate is taken at an offset each node computes for itself, the length of the prefix its own fifteen keys share, which is what keeps a lane discriminating on a keyspace whose shared bytes run deeper than a fixed window would skip. Section III-C measures the difference that makes. The descent hint is the fixed-window case and takes the first eight bytes outright, because it is compared across nodes rather than within one.

Aparajita differs from the rest of this line of work in what it is embedded in rather than in the kernel itself. These structures are read-optimised indexes built once or updated infrequently. A MemTable is written continuously by many threads while being read, is destroyed after a few seconds of life, and must allocate from a bump allocator that cannot free or reallocate. The kernel is the easy part; the concurrency protocol and the allocation discipline around it are where the design effort goes.

C. Key compression and surrogates

Compressing keys into fixed-width order-preserving codes is the subject of HOPE [15], and hybrid indexes [16] use compact representations for cold data. Our surrogate is a much simpler object, four bytes taken at a node-local offset, and it is a filter rather than a decision: every match it produces is confirmed against RocksDB’s comparator before it is used. Section III-C explains why removing that confirmation would lose keys rather than merely slow lookups down.

III. DESIGN

A. Node layout

A node holds sixteen 32-bit lanes in exactly one 64-byte cache line, aligned so it never straddles a line boundary. Fifteen of those lanes hold key surrogates and the sixteenth is a permanent sentinel, for the reason Section III-D gives; the node’s capacity is fifteen keys and we write it that way throughout. AVX2 covers the line in two 256-bit compares and AVX-512 in one, so a probe costs one line fill and a fixed number of instructions independent of where the answer lies. Figure 1 shows the two lines.

Everything else the node needs, the order word, the shared prefix length, the level-0 successor pointer and the key pointers, begins at offset 64. The separation is deliberate: a search that misses touches only the first line, and the cold fields are reached only after the vector step has narrowed the answer to a single candidate. The layout is enforced by static assertions rather than by comment, because the search kernel reinterprets the surrogate array as a vector and would silently misbehave if a field migrated into the first line.

B. Branchless ordered search

The kernel is where the branch mispredictions go away, and the reason it works for an ordered structure is a property of the mask rather than of the compare.

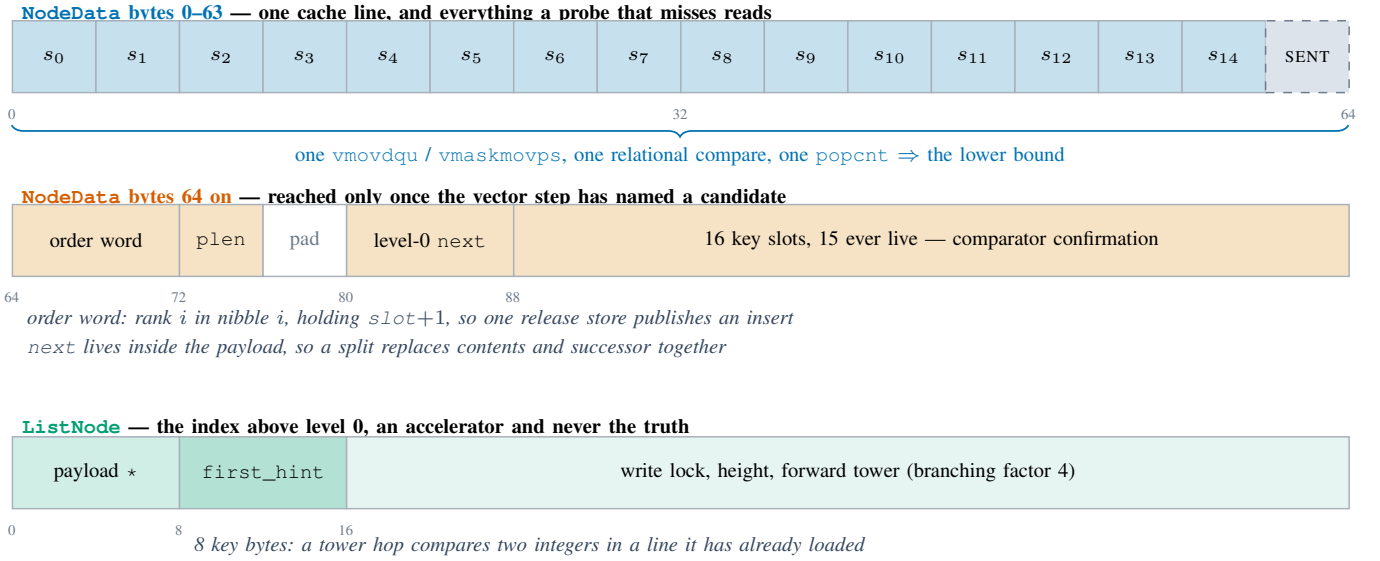


Fig. 1. Node layout, drawn to scale in bytes. The split at offset 64 is what makes a probe that finds nothing cost one line fill: the order word, the prefix length, the successor and the key pointers are all reached only once the vector step has narrowed the answer to a candidate. The successor and the order word live inside NodeData rather than beside the tower so that a split can replace contents and successor in one release store; moving either out would expose a state in which a key is duplicated across both halves or missing from both. Offsets are enforced by static assertion, since the search kernel reinterprets bytes 0–63 as a vector.

Let a node hold surrogates $s_0 \leq s_1 \leq \dots \leq s_{15}$ and let t be the target. A relational compare produces the bit mask $m_i = [s_i < t]$. Because the node is sorted, the set bits of m form a prefix, and the position of the first clear bit is therefore `popcount( $m$ )`. That value is exactly the lower bound: the number of keys in the node strictly below the target. One compare, one movemask and one population count give the answer with no branch and no trailing-zero count.

Sortedness is what makes the mask a prefix, but the kernel never reads the mask’s shape, only its population count. Section III-D uses that: the node the shipped kernel actually reads is *not* sorted in memory, and the same population count is still the same answer.

Two details in the kernel are necessary for correctness and easy to get wrong. AVX2 has no unsigned 32-bit compare, so `_mm256_cmpgt_epi32` is used with both operands biased by `0x80000000` to map unsigned order onto signed order. Without the bias the empty-slot sentinel `0xFFFFFFFF` reads as -1 and sorts below every real key, which inverts the padding that keeps a partially filled node sorted. AVX-512 provides `_mm512_cmplt_epu32_mask` natively and needs no bias.

Absence is reported as k , the node’s key count, rather than as -1 . The distinction is not stylistic. A -1 sentinel requires a data-dependent conditional to produce, and at a realistic hit ratio that conditional is unpredictable. Measured across that change on the same harness at an earlier revision, it cost 0.5 branch mispredictions per probe and roughly three quarters of the AVX2 kernel’s cycles, 25.4 against 6.2; both figures are in the artifact’s Phase 1 results rather than in Table I, which reports the current kernels only. Folding absence into

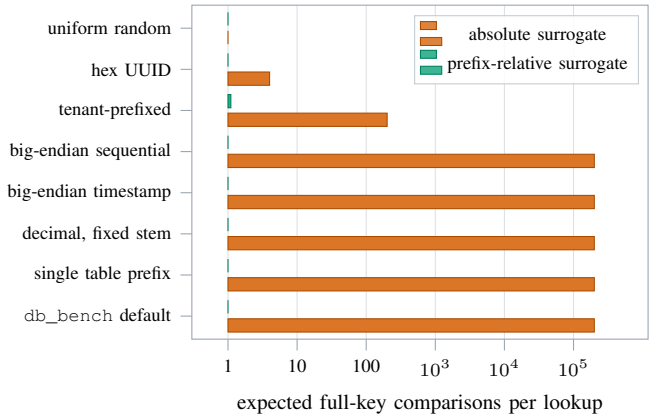


Fig. 2. Why the surrogate is taken after the node’s shared prefix, at $N = 200,000$ keys per distribution. A bar stopping at the 1 line is a perfect lane, one full-key comparison per lookup; a bar reaching N is a lane holding one value for the whole keyspace, which discriminates nothing and degenerates the vector step into a linear scan confirmed key by key. An absolute lane collapses on five of the eight distributions, including `db_bench`’s own key encoding, which is the keyspace every RocksDB measurement in this paper runs on. Only uniformly random binary keys leave it effective, and no production keyspace looks like that. Log scale; the absolute values are 1.0, 4.0, 201.0, and 200,000.0 exactly on the remaining five, and the prefix-relative values are 1.0 everywhere except tenant-prefixed, at 1.1.

the compare mask removes the conditional, and for the ordered kernel the convention falls out of the population count for free.

C. Prefix-relative surrogates

A vector lane holds 32 bits and a RocksDB key does not, so the lane holds a surrogate. The surrogate must preserve order, which rules out a hash: a hash distributes well and destroys the property the lower bound depends on. So the lane holds four

key bytes loaded big-endian, which makes an unsigned 32-bit compare of two surrogates reproduce RocksDB’s default bitwise memcmp order. Keys shorter than four bytes pad with 0x00, because a prefix sorts before any key extending it.

Taking those four bytes from the start of the key does not work, and measurement is what established it. Figure 2 reports the expected number of full-key comparisons a lookup performs, over eight key distributions at 200,000 keys each. An absolute surrogate takes a single value across the entire keyspace in five of them, because a table name prefix, a tenant identifier or a big-endian timestamp puts identical bytes at the front of every key in a database. A lane holding one value for every key discriminates nothing, and the SIMD kernel degenerates into a linear scan confirmed key by key. Only uniformly random binary keys leave an absolute lane effective, and no one writes those.

The fifth of the five deserves naming, because it is this paper’s own evaluation keyspace. `db_bench` writes the key ordinal big-endian into the first eight bytes of a sixteen-byte key and pads the rest, so at the two million keys of Section V the ordinal never reaches 2^{21} and the leading five bytes are zero on every key in the database. Every RocksDB figure we report is measured on a keyspace where an absolute surrogate would have held one value throughout. That is the hardest case for the lane rather than the easiest, which is worth stating plainly: the read results in Section V are not obtainable without the correction described next.

The correction is that the lane never needed to be order-preserving globally, only within a node. Each node stores the length of the prefix shared by all its keys, and surrogates are taken immediately after it. Two consequences live in the code. The prefix is recomputed from the first and last key whenever a node splits, since an insert at either end can shorten it and a stale prefix mis-orders every surrogate in the node. And a key that does not share the node’s prefix has no meaningful surrogate at that offset, so it is placed by full comparison instead; such a key is provably outside the node’s range rather than inside it, so its lane saturates in the direction it sorts. The right-hand column of The second series in Figure 2 is that design, and it restores every distribution to an effective lane, the worst at 1.1 comparisons per lookup.

The surrogate is a candidate and never an answer. Byte-wise order on the user key is not RocksDB’s order in general: a column family may install a custom comparator, and two entries whose user keys are equal are ordered by a *descending* sequence number packed into the internal key. Every SIMD candidate is therefore confirmed against the comparator, with a binary search fallback when confirmation fails. Four deliberately introduced kernel bugs passed a full differential test against RocksDB’s skip list because of this confirmation, which is why the kernels also carry direct unit tests against a scalar reference.

D. Append-only nodes and the order word

An ordered node must stay sorted, and an earlier version of this design kept it sorted by rebuilding it: every insert

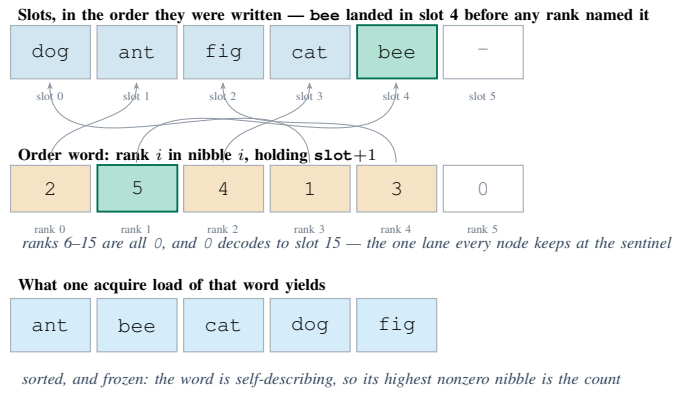


Fig. 3. An insert as two stores into a free slot and one release store of the order word. Slot order is arrival order and is never rearranged, so nothing already visible moves and a reader that takes one acquire load of the word holds a consistent node: everything the word names is frozen. A nibble holds `slot+1` rather than the slot, which costs one of the sixteen slots and buys two things — the word is self-describing, so no count needs a second store beside it, and an unused rank decodes to slot 15, the one lane every node keeps at the sentinel, so a short node is padded for free. The lower-bound kernel reads only the count from this word: a population count over a set does not depend on how the set is arranged, which is why the shipped kernel never undoes the permutation.

allocated a fresh payload, merged the new key into the sorted run, and published the new payload pointer with one release store. Publication was atomic and cheap to reason about. It also charged a whole node to the arena for every key written, 445.4 bytes per key against a live structure of 40, and scattered the nodes a lookup walks across ten times the memory they occupy.

The current design stops moving keys. A slot is written once and never again, and what gets republished is the *order* over the slots. Sorted rank i is held in nibble i of a 64-bit word, so a new order is one aligned 64-bit store and publication stays exactly as atomic as swapping a pointer was. An insert is two stores into a free slot, then one release store of the new order word. Nothing already visible moves, so a reader that takes one acquire load of the order word holds a consistent view: everything the word names is frozen.

A nibble holds `slot + 1` rather than the slot, so zero means “this rank is unused”. That costs one of the sixteen slots and buys two properties. The word becomes self-describing, since the count is the position of its highest nonzero nibble, so nothing needs a second store beside it; a second field would need a second store and would reopen the torn states the single store exists to prevent. And an unused rank decodes to slot 15, the lane no key occupies and every node keeps at the sentinel, so a node is padded above its last key for free.

Slot order is insertion order, so lanes are not sorted in memory. The ordered kernel turns out not to care. The lower bound is a count of the live keys below the target, and a count over a set does not depend on how the set is arranged, so permuting the lanes into rank order produces a different vector and the same population count. An earlier version of this design did permute, at a measured cost of 21.03 against

9.89 cycles per probe on AVX2, and removing the permutation changed no answer.

What the kernel does still need is a mask over the live lanes, because a writer may be storing into the slot at index *count* while the kernel runs and an unmasked 64-byte load would race with it. That mask is the entire residual cost of the append-only layout, $1.50\times$ on AVX-512 and $1.94\times$ on AVX2 against the same kernel over a sorted node. It is justified by the memory model [17] alone: no test detects its removal, and ThreadSanitizer cannot either, because the compiler does not instrument the vector load. The mask is a correctness device and not a performance hint, because masked lanes in `vmaskmovps` and `vmoaddqu32` are architecturally not accessed at all; the unmasked load would be a real data race that our tests happen not to observe.

E. Descent hints

Reaching the right node is a skip-list tower above level 0, with branching factor four. Everything in the tower is an accelerator and never the truth, because the search only follows a pointer to a node whose first key is at or below the target, and nodes are ordered and never removed; a stale or missing tower pointer costs a longer walk and never a wrong answer.

The descent was the dominant cost of a lookup and its size had been estimated rather than measured. Measured, a lookup over 500,000 keys examined 30.0 successors, not the eight an earlier design note had assumed, and each one was a chain of dependent loads ending in a virtual comparator call on a full internal key. Each node now caches the first eight bytes of its first key in its header, so a hop compares two integers in a line the hop has already loaded, and only a tie falls through to the comparator.

Three invariants hold this up. The hint is immutable for every node but the head, because the descent returns the last node whose first key is at or below the key being inserted, so an insert into a non-head node sorts at position 1 or later and a split leaves the left half's first key alone; the head is the exception and is also the only node no hop ever moves to, so its hint is never read. A tie decides nothing and falls through to the comparator, which is why the hint is eight bytes and the surrogate four: a surrogate tie costs one comparison inside a node already in cache, and a hint tie costs the miss chain the hint exists to avoid. And the hint is stored before the payload, because a first key never rises, so a new hint beside an old payload is a hint that is too small, and too small only ever lets a hop enter a node it was already entitled to enter. The other order would expose a hint that is too large, which would make a hop stop one node short of the key it wanted.

The fast path is disabled unless the representation can confirm that the user comparator agrees with bitwise order, because a wrong hop is not a slow lookup but a lost key.

F. Concurrency

Readers are lock-free and never write. A reader takes one acquire load of the order word and reads only slots that word names, all of which are frozen.

Writers take a per-node spin lock that pauses 64 times and then yields. This is a deliberate narrowing of the lock-free goal [18] and we state it rather than hide it. A fully lock-free ordered insert has to shift keys inside a sorted node, which is not one atomic operation, so the design publishes through a single release store with a retry and uses a lock to stop writers livelocking against each other. Contention is per node, so two writers meet only when they target the same fifteen keys. The backoff yields rather than spinning flat because RocksDB routinely runs more writers than cores, where a flat spin burns every other thread's time slice while the lock holder waits to be scheduled.

A split is the one operation that changes a node's contents and its successor at the same time, so the level-0 link and the order word both live inside the payload. The right half is published as a new node first, then a single release store swaps the left half's payload pointer and its successor together. A reader sees either the old full node or the new pair, never a state where a key is duplicated across both or missing from both.

Section III-D reduced what the writer lock covers from an allocation and a fifteen-key merge to three stores, which makes replacing it with a compare-and-swap on the order word a live question. We did not attempt it here, for a reason Section V-H makes concrete.

IV. IMPLEMENTATION

Aparajita is a header-only C++20 library of roughly 2,000 lines plus a RocksDB plugin. The core is a template over a traits policy carrying the three things that differ between the standalone structure and the RocksDB representation: what an entry is, how order is decided, and where memory comes from. The split, the append path and the tower exist once and are shared by both.

The plugin derives from `MemTableRep` and `MemTableRepFactory` and builds through RocksDB's plugin/ mechanism, so `db_bench -memtablerep=aparajita` selects it without patching RocksDB sources. It implements `Allocate`, `InsertKey` and its concurrent variants, `Get`, `GetIterator`, `ApproximateMemoryUsage` and `MarkReadOnly`. `IsInsertConcurrentlySupported` returns true, which matters because RocksDB routes concurrent writes only to a representation that advertises support; without it every multi-threaded write benchmark serialises, which is part of why `VectorRep` loses on write throughput despite a friendlier layout.

Two integration details are worth recording. `MemTableRep::Allocate` hands key storage to the caller, so the representation never copies a key and an entry is a bare 8-byte pointer into RocksDB's arena. And the descent hint's fast path needs to confirm the comparator is the default bitwise one, for which `MemTableRep::KeyComparator` exposes no route other than a `dynamic_cast`; RocksDB compiles `Release` with `-fno-rtti` by default, so the evaluation builds with

USE_RTTI=ON, a supported CMake option applied to the whole tree. The skip-list rows in Section V are the control showing that flag alone moves nothing.

Correctness is checked by a differential test against RocksDB’s own skip list: forward and backward iteration, every Get, Seek and SeekForPrev agree byte for byte, under concurrent writes, across a flush, and under a reverse comparator. RocksDB’s own MemTable gtests build their options through CurrentOptions() and never consult -memtablerep, and db_stress resolves that flag through a hardcoded enum, so neither can be pointed at a plugin representation without patching RocksDB. That is a limitation of the harness rather than of the plugin, and it is why a differential test stands in for the upstream suite.

V. EVALUATION

A. Method

All figures come from a GCP c4-standard-24 in us-central1-b: a Xeon Platinum 8581C (Emerald Rapids), 12 physical cores and 24 logical with SMT on, 88 GiB, Ubuntu 24.04, GCC 14.2, RocksDB v9.11.2 built Release with USE_RTTI=ON. One host, one RocksDB tree, one set of build flags, one uninterrupted session, nothing else running. Every representation is selected through -memtablerep on the same db_bench binary, so the only thing differing between columns is the representation.

The workload is db_bench’s default 16-byte key and 100-byte value over a 2,000,000-key space, with compression off, automatic compaction off, the write-ahead log disabled except where Section V-E enables it, and a 4 GiB write buffer so that nothing flushes mid-run and the MemTable stays the object under test. Every run reports its SST count, and no run behind Tables II or V produced more than the single file the flush on close writes. Section V-F relaxes the write buffer to 64 MiB and is the one measurement here that flushes on purpose.

Five repetitions per cell. fillrandom runs in two independent passes, because the scaling pass and the read passes each fill first, so a fill cell carries ten samples and a read cell five.

Quota capped the shape at 24 vCPUs. The 1- and 4-thread points sit below physical core count, 16 sits between physical and logical, and 64 is $2.7\times$ oversubscribed. The top of every curve is contention rather than headroom and is labelled as such throughout.

Two methodological points cost us a full run each and generalise past this project.

A benchmark knob that sets both the work and the data has to be split before a curve over it means anything. db_bench’s -num is per thread and controls both how much work a thread does and the range its random keys are drawn from. The natural way to hold total work constant across thread counts, -num=TOTAL/threads, also divides the key space: at 1 thread the MemTable held 1.26 million distinct keys and at 16 threads it held 125,000, each written sixteen times, and the measured read hit rate drifted from 63% to 100%

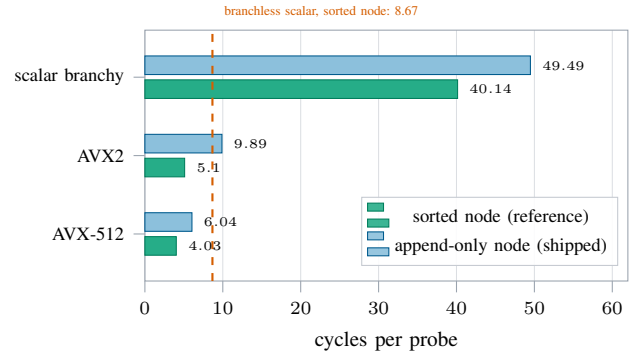


Fig. 4. Kernel cost over one 16-lane node, 200,000 probes, median of nine repetitions. The vector kernels retire zero branch mispredictions per probe at any hit ratio; the branchy scalar baselines retire about one. The gap between the two series is not slack but the live-lane mask, which exists because a writer is storing into the slot the kernel is reading. The dashed line is a branchless scalar lower bound over a sorted node: on an AVX2-only host the shipped kernel clears it by less than a cycle, which is the honest statement of what SIMD buys on that ISA.

across the curve. That is a working-set sweep in a thread-scaling curve’s clothes, and it flattered the reads. In the figures reported here -num is pinned at 2,000,000 and -writes and -reads carry the per-thread work, holding the hit rate at 63% everywhere.

Counter differencing has to cancel the setup exactly, not approximately. perf wraps a whole process, so a read benchmark’s counters include the fill that populated the MemTable, the database open and the flush on close. Subtracting a separate fill-only run fails, because the fill is twenty times the size of the read pass and the subtraction is two large noisy numbers differenced to a small one; it produced negative instruction counts. Every counter figure below is instead the difference between two runs of the *same* benchmark that differ only in operation count, with an identical fill at an identical seed, so everything that is not the extra operations cancels exactly. Reads difference 20,000,000 against 2,000,000 and writes 2,000,000 against 200,000.

B. Search kernels in isolation

Table I reports the kernels over 200,000 probes at a 0.5 hit ratio, nine repetitions, median, with hits landing on a uniformly random slot and the hit and miss mix randomised rather than blocked. Blocking the mix or always probing slot 0 lets the branchy baseline exit after one comparison and hides exactly the mispredictions the design removes.

The branch behaviour is the cleanest result in the paper. Every vector kernel and every branchless scalar kernel records 0.0000 mispredictions per probe, against 0.5253 for the branchy scalar baseline, and the vector kernels do not respond to the hit ratio at all: at a 0.9 hit ratio every ordered row reproduces to within 0.02 cycles, while the branchy baseline moves from 28.49 to 40.47.

Two rows deserve a reader’s scepticism and we state them before drawing any conclusion from the table.

TABLE I
SEARCH KERNELS, CYCLES PER PROBE, HIT RATIO 0.5

kernel	cyc/probe	ins/probe	br/probe	miss/probe
<i>equality search over a sorted node</i>				
scalar linear	27.65	34.50	15.24	0.5253
scalar binary	70.24	51.41	13.16	1.9095
scalar branchless	16.53	90.00	3.00	0.0000
AVX2	5.01	20.00	3.00	0.0000
AVX-512	4.04	15.00	3.00	0.0000
<i>lower bound over a sorted node</i>				
scalar	40.14	26.69	11.33	1.0390
scalar branchless	8.67	44.00	3.00	0.0000
AVX2	5.10	25.00	3.00	0.0000
AVX-512	4.03	16.00	3.00	0.0000
<i>lower bound over an append-only node (shipped)</i>				
scalar	49.49	87.35	21.66	0.9276
AVX2	9.89	41.00	4.00	0.0000
AVX-512	6.04	30.00	4.00	0.0000

The shipped AVX2 ordered kernel, at 9.89 cycles per probe, is 14% slower than scalar branchless lower bound at 8.67. SIMD losing to scalar on the ISA that ships would be a serious problem if the scalar kernel were an available alternative, and it is not: it reads a sorted node, and no scalar branchless kernel exists for the append-only layout. The comparison the design actually faces is 9.89 against 49.49. The wider point stands, which is that the append-only layout bought its write win at a read-side price of $1.94\times$ on AVX2 and $1.50\times$ on AVX-512, and that price is the live-lane mask described in Section III-D.

The equality rows for AVX2 and AVX-512 are not stable across our runs. A second run on a different instance size of the same part reports 4.02 and 5.02 rather than 5.01 and 4.04, inverting which ISA is faster while retained instruction counts stay at 20 and 15. The ordered families reproduce across those same two runs to within 0.2 cycles, so we quote the ordered rows and make no claim about the relative speed of the two ISAs on equality search.

C. Point lookups

Table II reports `readrandom` over a MemTable-resident working set with the write-ahead log disabled, and `readwhilewriting` with one writer alongside the readers. No run in either table produced more than the single SST file that the flush on close writes, so every figure is served by the MemTable.

Aparajita is ahead at every thread count on both benchmarks, and at seven of the eight points the samples do not overlap at all: Aparajita’s slowest of five runs beats the skip list’s fastest. The exception is `readwhilewriting` at 16 threads, where one Aparajita run came in at 6,082 kops/s against a skip-list best of 7,019, which is also the widest range in the table at 31.8%. The other seven cells separate cleanly, and the two benchmarks agree with each other at every thread count.

Table III gives the marginal per-operation cost from the two-point differencing described above. Aparajita retires 3,271.5

TABLE II
READ THROUGHPUT, KOPS/S, WAL OFF. MEDIAN OF FIVE RUNS; THE RNG COLUMN IS (max – min)/median OVER THOSE FIVE. VECTORREP IS FOUR ORDERS OF MAGNITUDE SLOWER AND IS REPORTED IN SECTION V-G.

threads	Aparajita	rng	skip list	rng	vs. skip list
<i>readrandom</i>					
1	731.9	7.8%	528.2	17.7%	+38.6%
4	2703.1	3.0%	1969.1	5.2%	+37.3%
16	8773.9	8.4%	7181.8	23.8%	+22.2%
64	10913.5	4.7%	8677.9	9.8%	+25.8%
<i>readwhilewriting</i>					
1	705.6	2.6%	519.7	4.4%	+35.8%
4	2693.8	2.7%	1950.2	6.9%	+38.1%
16	8691.9	31.8%	6738.8	8.4%	+29.0%
64	10531.9	7.8%	8578.5	13.3%	+22.8%

TABLE III
MARGINAL COST PER OPERATION, 16 THREADS

benchmark	rep	instr	br miss	br miss %	L1 miss
<i>fillrandom</i>	Aparajita	22181.7	16.15	0.361	178.86
	skip list	24177.1	18.01	0.379	227.73
	VectorRep	14989.5	50.70	1.636	176.96
<i>readrandom</i>	Aparajita	3271.5	8.46	1.241	62.18
	skip list	7349.9	10.39	0.727	103.99
<i>readwhilewriting</i>	Aparajita	3622.2	9.53	1.265	72.41
	skip list	7597.6	11.93	0.808	110.70

instructions per lookup against the skip list’s 7,349.9, and takes 62.18 L1 misses against 103.99. Those are the two mechanisms the design targets, showing up where they were predicted: fewer instructions from replacing a chain of virtual comparator calls with integer compares in already-loaded lines, and fewer L1 misses from a structure that consumes 100.6 rather than 445.4 bytes of arena per key.

One inversion in that table is worth stating rather than burying. Aparajita’s branch miss *rate* on reads is worse than the skip list’s, 1.241% against 0.727%, while its branch misses *per operation* are better, 8.46 against 10.39. Both are true, and they are consistent: the design executes less than half as many branches per lookup, so the same absolute count of mispredictions is a larger share of a much smaller denominator. Per-operation cost is what a workload pays. A paper quoting the rate alone would be quoting the single framing in which this design looks worse than the structure it replaces.

D. Ordered access

Section II-A argues that the design has to be ordered because a MemTable iterator has to be, and Section V-G measures what giving that up costs VectorRep. Table IV measures what the ordering buys and what it costs, and the answer divides at the boundary between finding a position and walking from it.

A Seek on its own is 15% to 29% faster than the skip list at every thread count, narrowing as threads rise exactly as the point lookups do, and for the same reason: a Seek is a descent

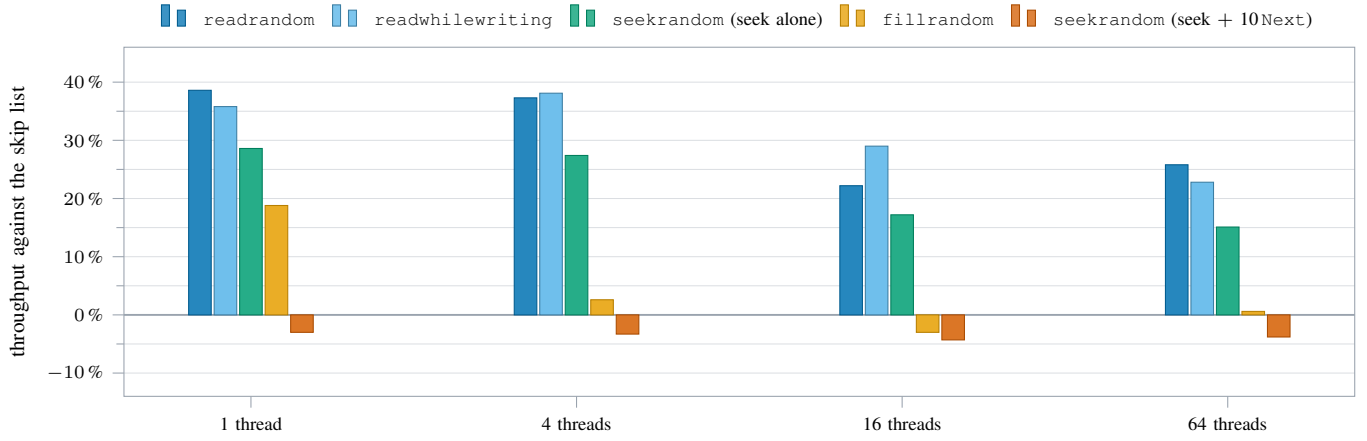


Fig. 5. Every `db_bench` result in this paper, as throughput against RocksDB’s default skip list on the same host and the same build. Positive is faster. The three read workloads win at every thread count with non-overlapping samples, and the two that do not are the ones the design pays for: iteration, where the skip list’s `Next` is one pointer load and ours decodes a rank, and multi-threaded `fillrandom`, where neither representation scales because RocksDB’s write-thread group dominates both (Section V-E). Read cells are medians of five runs, fill cells of ten; the underlying numbers are in Tables II–V.

TABLE IV
SEEKRANDOM THROUGHPUT, KOPS/S, WAL OFF, OVER A RESIDENT MEMTABLE. MEDIAN OF FIVE RUNS. THE `NEXTS` COLUMN IS THE NUMBER OF `NEXT` CALLS ISSUED AFTER EACH `SEEK`.

threads	Aparajita	skip list	vs. skip list
Seek alone, nexts = 0			
1	428.2	333.0	+28.6%
4	1744.3	1368.8	+27.4%
16	5688.7	4853.0	+17.2%
64	7639.7	6636.1	+15.1%
Seek then ten Next, nexts = 10			
1	191.7	197.6	−3.0%
4	744.1	769.6	−3.3%
16	2246.6	2348.0	−4.3%
64	3093.8	3217.5	−3.8%

TABLE V
FILLRANDOM THROUGHPUT, KOPS/S, WAL OFF

threads	Aparajita	skip list	VectorRep	vs. skip list
1	678.9	571.2	2222.9	+18.8%
4	677.9	660.9	1201.0	+2.6%
16	1394.8	1437.7	911.1	−3.0%
64	505.8	502.6	1101.0	+0.6%
scaling relative to 1 thread				
4	1.00×	1.16×	0.54×	
16	2.05×	2.52×	0.41×	
64	0.75×	0.88×	0.50×	

paid per step.

followed by a lower bound, which is what the hints and the kernel make cheap.

A `Seek` followed by ten `Nexts` is 3% to 4% *slower*. The two results do not conflict. A skip-list `Next` is one pointer load along the level-0 chain. Aparajita’s decodes the next rank from the order word, indexes the slot that nibble names, and follows a key pointer into the arena, which is more work per step for precisely the reason the append-only layout is cheap to write: the sorted order is a computed permutation and not the physical one. At ten steps per `Seek` the walk dominates the descent and the design is behind. The margin is small and the ranges overlap at 1, 4 and 64 threads; at 16 they do not, and the sign is the same at all four points, so we report a loss rather than a tie.

That bounds the claim this paper makes. Aparajita is a faster place to look a key up and a slightly slower place to scan from, and a workload dominated by long range scans over the MemTable is not one these numbers support. The fix has an obvious shape and we did not attempt it: the order word already sits in a line the iterator has loaded, so a `Next` that decoded several ranks at once would amortize what is currently

E. Inserts, and the ceiling `db_bench` imposes

Table V reports `fillrandom`. Aparajita is 18.8% ahead at one thread, 2.6% ahead at four, and tied at 16 and 64. The 4-thread figure reproduces an independent 2.0% measured on a different host in an earlier phase. The ties are in the samples and not a rounding of the medians. At 16 threads the two independent fill passes disagree about the sign, giving −3.0% and +0.0%, and the ranges overlap outright: 1387.9–1430.1 kops/s for Aparajita against 1397.2–1450.1 for the skip list. At 64 threads both lie inside a 1% band, 503.2–507.8 against 500.2–504.3.

The scaling rows are the tell. Neither representation scales, and two structures with entirely different insert paths do not accidentally share a scaling curve. The counters name the cause: a `fillrandom` operation at 16 threads retires 22,181.7 instructions in Aparajita and 24,177.1 in the skip list. A MemTable insert is not twenty-two thousand instructions; an append into a node under a spin lock is a few hundred. The rest is RocksDB’s write-thread group, where writers spin waiting for a group leader to commit the batch, charged identically to every representation. At 16 threads on 12 cores,

TABLE VI

ARENA COST MADE VISIBLE: 64 MiB WRITE BUFFER, ONE THREAD, 2,000,000 KEYS. A REPRESENTATION CHARGING MORE ARENA PER KEY FITS FEWER KEYS INTO A MEMTABLE AND FLUSHES MORE OFTEN.

rep	fill $\mu\text{s/op}$	L0 files	keys/flush	flush $\mu\text{s/key}$
Aparajita	1.031	7	285,714	0.425
skip list	1.313	5	400,000	0.441

`db_bench fillrandom` measures RocksDB’s write path with the MemTable as a rounding error.

`memtablerep_bench` has no write group and no SST, block cache or version machinery, and the difference reappears intact. Insert costs $0.5435 \mu\text{s/op}$ against the skip list’s 0.8040 , the skip list 48% slower, and the gap holds at 48% to 52% across all four thread counts. RocksDB hardcodes that benchmark’s `fillrandom` to a single thread regardless of `-num_threads`, so those four columns are four samples of one single-threaded measurement rather than a curve; they agree to within 3%, which is the useful thing they say. Its `readrandom` narrows under oversubscription, from 62.5% at one thread to 10.4% at 16 and 0.4% at 64, which looks like a contradiction of Table II’s +25.8% at 64 threads in the harness with more machinery in it. The absolute figures reconcile the two. At 64 threads `memtablerep_bench` costs 52.2 and $52.5 \mu\text{s}$ per lookup for the two representations, against `db_bench`’s 4.4 and 5.7: the rep-level harness has degraded a hundredfold from its own single-threaded $0.50 \mu\text{s}$ while `db_bench` degrades threefold. A lookup costing on the order of a microsecond is a fiftieth of a $52 \mu\text{s}$ budget, so at 64 threads that benchmark has stopped resolving the representation rather than the representations having converged. We read its 1- and 4-thread points and not its 64-thread one, and the read claim in this paper rests on Table II.

The claim this evidence supports is that the representation inserts roughly half again as fast as the skip list, visible undiluted at one thread and in `memtablerep_bench`, and swamped by RocksDB’s write-group machinery in multi-threaded `db_bench`. It does not support a multi-threaded `db_bench` write scaling claim, and we do not make one.

The write-ahead log behaves as expected and we measured it rather than assuming it. At 16 threads with the log enabled the two representations run at 1043.0 and 1048.1 kops/s against 1394.8 and 1437.7 with it disabled: the log costs about 27% and closes the already-small gap to 0.5%.

F. Arena cost, and what it costs to flush

Every figure above holds the MemTable resident, which is what isolates the representation and is also what hides its one clear regression. Aparajita charges about 1.4 times the skip list’s arena per key. Table VI makes that visible by shrinking the write buffer to 64 MiB and letting the same two million keys flush: Aparajita fits 285,714 keys into a MemTable against the skip list’s 400,000 and writes seven L0 files where the skip list writes five.

Flushing is not what those extra files cost. The same two million keys are written either way and the per-key flush cost is within four percent, 0.425 against $0.441 \mu\text{s}$, so the total time spent flushing is if anything slightly lower. Walking the structure, which is what a flush does, is not where the arena overhead lands, and the fill itself is 21% faster here because this configuration is the one where the write buffer is small enough for the insert path to matter.

What the overhead costs is L0 files, forty percent more of them for the same data. That is compaction work, which this configuration disables and therefore does not measure, and it is also more frequent write stalls at any realistic `level0_slowdown_writes_trigger`. We state it as an open cost rather than estimate it.

G. VectorRep

VectorRep has the friendliest memory layout of the three and loses anyway, which is the comparison that motivates an ordered design. It is by far the fastest single-threaded fill at 2,222.9 kops/s, because appending to a vector is the cheapest insert available, and it is the only representation whose fill throughput *falls* as threads are added, $0.54\times$ at 4 and $0.41\times$ at 16, because `IsInsertConcurrentlySupported` returns false and every write serialises.

Its reads are not slow so much as disqualifying: 0.9 to 3.3 seconds per lookup, and 6.4×10^9 retired instructions per lookup. `VectorRep::Get` builds an iterator, and on a mutable MemTable that iterator copies the entire bucket and sorts the copy, because a sorted order cannot be cached on a structure still being appended to. Its read benchmarks are capped at 20 operations for this reason and reported as capped; uncapped they would have run for days.

H. Race freedom

The concurrent insert and read test runs under ThreadSanitizer [19] at 1, 4, 16 and 64 threads, and reports no races at any thread count on any run.

The cost of that instrumented run is heavy-tailed in a way worth recording, because we published a wrong figure for it before measuring it properly. Five runs of the same binary on one idle host gave totals of 9.4 s, 186 s, 298 s, 373 s and 524 s for identical work, and a single 6.9 ms sample from the fast tail had been reported as the result. The per-phase breakdown locates it: 640 total inserts take under 3 ms of race time at 4 threads and up to nine minutes at 64. What grows is the number of threads spinning in an instrumented `test_and_set` loop, where ThreadSanitizer’s per-thread vector clocks make each of those atomics cost $O(\text{threads})$.

That is an artifact of instrumentation and not evidence about uninstrumented throughput, and we are explicit that it is not a reason to believe the spin lock is expensive in production. It is a reason to want the compare-and-swap formulation, since that removes the loop the instrumentation chokes on.

VI. LIMITATIONS

Three exit criteria this work set itself are not fully met, and one of them concerns the mechanism the design is most directly about.

Cross-core invalidation is unmeasured. Loads that hit a line another core holds modified are the counter that would speak most directly to a cache-line argument, and we could not collect it. The cloud provider’s API rejected the vPMU tier documented to cover LLC events on both API versions tried. Under the tier it did accept, `mem_load_l3_hit_retired.xsnf_fwd` enumerates in `perf list`, programs without error, reports 100% enabled time, and then counts exactly zero against a run retiring 1.08×10^{11} instructions across 16 threads. `cache-references` and `cache-misses` return *not supported* outright. These are PEBS-backed events and the hypervisor does not implement PEBS [20], so the counter reads zero rather than refusing.

That failure mode is worse than an unsupported event, because “0.000 cross-core hits per operation” is a publishable-looking number and it would have been false. Our harness now calibrates the event against a workload that must produce cross-core sharing and demotes it to unavailable if it stays at zero. Two related traps are worth passing on: Intel retired the `XSNP_HITM` name at Ice Lake and the counterpart on this part is `XSNP_FWD`, so resolving the event name against the host’s own `perf list` matters; and `perf list | grep -q` makes `grep` exit at its first match, `perf list` then dies of SIGPIPE, and under `set -o pipefail` every event the host has is reported absent. Closing this needs bare metal or a provider that exposes PEBS to guests.

The scaling curve has no headroom. Quota capped the host at 12 physical cores, so every point above 12 threads is contention. That regime is arguably the one RocksDB runs in, since more writers than cores is normal, which is why the per-node backoff yields rather than spinning flat. It still cannot be read as scaling, and the shape of the curve on a machine with 64 real cores is unknown.

The write argument depends on a benchmark most readers do not use. Because multi-threaded `db_bench fillrandom` measures RocksDB’s write path rather than the representation, the insert win has to be argued from `memtablerep_bench` and from the single-threaded point.

Two cache effects adjacent to the argument are unmeasured. The design aligns to 64 bytes and pads `ListNode` to a full line so that writers to different nodes do not share one. Intel parts also run an adjacent-line prefetcher that pulls lines in pairs, so two nodes landing on lines N and $N+1$ can still be invalidated together under concurrent writers, and we did not isolate that: it would need the same cross-core counter that came back dead, which is why it is a gap rather than an oversight. Separately, the host is a single socket, so nothing here says how the structure behaves when a writer’s arena block is resident on another NUMA node. Neither effect is under the representation’s control in RocksDB, where node

memory comes from `ConcurrentArena`, but both would matter on a two-socket machine.

The consequences of flushing more often go unmeasured. The arena overhead in Section V-F produces forty percent more L0 files for the same data. Flush time absorbs none of that, but compaction and the write stalls that L0 file counts trigger would, and every configuration here disables automatic compaction so that the MemTable is what is being measured. An end-to-end comparison under compaction is the natural next measurement and would be the first place this design could lose overall while winning on every number reported above.

Two smaller omissions are deliberate. The representation does not implement the prefix-extractor and Bloom filter paths [21], [22], so it inherits the base class’s full iterator, which is correct but gives up the skip list’s prefix-Bloom short circuit. And AVX-512 is opportunistic behind a CPUID check rather than required, on availability grounds rather than on the frequency behaviour the optimization manual describes for wide vector work [23]: AVX-512 is absent from most AMD parts before Zen 4, and Intel disabled it in client parts after Rocket Lake.

VII. CONCLUSION

Replacing a pointer-chasing skip list with cache-line nodes searched by a branchless SIMD kernel improves MemTable point lookups by 22% to 39% inside RocksDB, with the improvement traceable to 55% fewer retired instructions and 40% fewer L1 misses per lookup. The improvement is in finding a key and not in walking from one: an ordered seek is 15% to 29% faster and a seek followed by ten iterator steps is 3% to 4% slower, because the sorted order over an append-only node is a permutation to be decoded rather than a chain to be followed. The same trade shows up in space, where the representation charges 1.4 times the skip list’s arena per key. The kernel work is necessary and far from sufficient: two structural problems outside the kernel, an expensive tower descent and a per-insert node rebuild, each cost more than the kernel saved, and the design only became competitive once both were fixed.

The result we did not get is as informative as the ones we did. Multi-threaded `db_bench` inserts are bounded by RocksDB’s write-thread group at over 22,000 instructions per operation, which no MemTable design can move, and the counter that speaks most directly to cross-core invalidation could not be collected on virtualised hardware at all. Both are properties of the measurement environment rather than of the structure, and both are the kind of thing that is easier to state than to discover.

AVAILABILITY

Source, measurement scripts and the raw output behind every figure are at <https://github.com/sinhaparth5/aparajita-memtable>, under Apache 2.0 or MIT at the user’s option. The revision these results were produced from is archived at <https://doi.org/10.5281/zenodo.22180369> [24]; the

concept identifier 10.5281/zenodo.22180368 resolves to the current version.

[24] P. K. Sinha, “Aparajita: A SIMD MemTable for LSM-tree key-value stores,” Version 0.5.0, Zenodo, <https://doi.org/10.5281/zenodo.22180369>, 2026.

REFERENCES

- [1] P. O’Neil, E. Cheng, D. Gawlick, and E. O’Neil, “The log-structured merge-tree (LSM-tree),” *Acta Informatica*, vol. 33, no. 4, pp. 351–385, 1996.
- [2] C. Luo and M. J. Carey, “LSM-based storage techniques: A survey,” *The VLDB Journal*, vol. 29, no. 1, pp. 393–418, 2020.
- [3] S. Dong, A. Kryczka, Y. Jin, and M. Stumm, “RocksDB: Evolution of development priorities in a key-value store serving large-scale applications,” *ACM Transactions on Storage*, vol. 17, no. 4, pp. 1–32, 2021.
- [4] S. Dong, M. Callaghan, L. Galanis, D. Borthakur, T. Savor, and M. Strum, “Optimizing space amplification in RocksDB,” in *Conference on Innovative Data Systems Research (CIDR)*, 2017.
- [5] Meta Platforms, “RocksDB: A persistent key-value store for fast storage environments,” <https://rocksdb.org>, 2025.
- [6] Google, “LevelDB: A fast key-value storage library,” <https://github.com/google/leveldb>, 2025.
- [7] W. Pugh, “Skip lists: A probabilistic alternative to balanced trees,” *Communications of the ACM*, vol. 33, no. 6, pp. 668–676, 1990.
- [8] J. Rao and K. A. Ross, “Making B+-trees cache conscious in main memory,” in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 2000, pp. 475–486.
- [9] C. Kim, J. Chhugani, N. Satish, E. Sedlar, A. D. Nguyen, T. Kaldewey, V. W. Lee, S. A. Brandt, and P. Dubey, “FAST: Fast architecture sensitive tree search on modern CPUs and GPUs,” in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 2010, pp. 339–350.
- [10] B. Schlegel, R. Gemulla, and W. Lehner, “k-ary search on modern processors,” in *Proceedings of the Fifth International Workshop on Data Management on New Hardware (DaMoN)*, 2009, pp. 52–60.
- [11] S. Zeuch, J.-C. Freytag, and F. Huber, “Adapting tree structures for processing with SIMD instructions,” in *Proceedings of the 17th International Conference on Extending Database Technology (EDBT)*, 2014, pp. 97–108.
- [12] Y. Mao, E. Kohler, and R. T. Morris, “Cache craftiness for fast multicore key-value storage,” in *Proceedings of the 7th ACM European Conference on Computer Systems (EuroSys)*, 2012, pp. 183–196.
- [13] V. Leis, A. Kemper, and T. Neumann, “The adaptive radix tree: ARTful indexing for main-memory databases,” in *IEEE 29th International Conference on Data Engineering (ICDE)*, 2013, pp. 38–49.
- [14] J. J. Levandoski, D. B. Lomet, and S. Sengupta, “The Bw-tree: A B-tree for new hardware platforms,” in *IEEE 29th International Conference on Data Engineering (ICDE)*, 2013, pp. 302–313.
- [15] H. Zhang, X. Liu, D. G. Andersen, M. Kaminsky, K. Keeton, and A. Pavlo, “Order-preserving key compression for in-memory search trees,” in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 2020, pp. 1601–1615.
- [16] H. Zhang, D. G. Andersen, A. Pavlo, M. Kaminsky, L. Ma, and R. Shen, “Reducing the storage overhead of main-memory OLTP databases with hybrid indexes,” in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 2016, pp. 1567–1581.
- [17] H.-J. Boehm and S. V. Adve, “Foundations of the C++ concurrency memory model,” in *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2008, pp. 68–78.
- [18] M. Herlihy and N. Shavit, *The Art of Multiprocessor Programming*. Morgan Kaufmann, 2008.
- [19] K. Serebryany and T. Iskhodzhanov, “ThreadSanitizer: Data race detection in practice,” in *Proceedings of the Workshop on Binary Instrumentation and Applications (WBIA)*, 2009, pp. 62–71.
- [20] *Intel 64 and IA-32 Architectures Software Developer’s Manual, Volume 3B: System Programming Guide*, Intel Corporation, 2024.
- [21] B. H. Bloom, “Space/time trade-offs in hash coding with allowable errors,” *Communications of the ACM*, vol. 13, no. 7, pp. 422–426, 1970.
- [22] N. Dayan, M. Athanassoulis, and S. Idreos, “Monkey: Optimal navigable key-value store,” in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 2017, pp. 79–94.
- [23] *Intel 64 and IA-32 Architectures Optimization Reference Manual*, Intel Corporation, 2024.